AdaBoost for regression

AdaBoost.R2: weighted regression trees, weighted median

Data: eight points

Summary.
1. Give every data point a *weight*; at the start all points matter equally.
2. Grow a small tree that predicts y, paying more attention to the heavier points.
3. Measure how wrong the tree is on each point, *relative to its worst mistake*.
4. Make the points the tree predicted **well** count **less** (and so, relatively, the badly predicted ones count more), then grow the next tree.
5. The final prediction is the **weighted median** of all the trees' predictions, where a tree that did better gets a louder voice.

Colour key: every symbol keeps its colour in every formula

y	**the truth**	the observed value (minutes per epoch)
h	**the tree's prediction**	$h(x)$: the average of y in the leaf that x reaches
w	**the weight of a point**	how much the point counts; they add to 1
δ	**the error of a point**	$\delta_i = \lvert y_i - h(x_i)\rvert$
Δ	**the largest error**	the biggest δ_i in this round
ℓ	**the relative loss**	$\ell_i = \delta_i/\Delta$, between 0 (perfect) and 1 (worst)
β	**the shrink factor**	$\beta = \bar{\ell}/(1-\bar{\ell})$, below 1 when the average loss $\bar{\ell}$ is below $\frac{1}{2}$
α	**the confidence**	$\alpha = \ln(1/\beta)$: how much the tree's opinion counts in the final median

Rounds and trees. One **round** means: grow *one* tree on the current weights, measure its loss, work out its confidence α, and change the weights. So the number of trees always equals the number of rounds: after 3 rounds there are 3 trees. We stop after T rounds; here $T = 6$, so there are six trees. (T is our choice. The algorithm also stops early if a tree is no better than a coin toss, $\bar{\ell} \geq \frac{1}{2}$. In practice T is chosen on separate validation data.) Do not confuse the rounds with the questions *inside* one tree: a single tree here asks three questions (one at the top, one in each branch) and has four leaves.

1. The problem and the data

Regression means predicting a number. Here is a small, *made-up, illustrative* data set: how long one training epoch takes (minutes) for a neural network of a given size. (The same kind of data as in Part 1, with different numbers.)

point i	model size x (millions of parameters)	time per epoch y (minutes)
1	1	1.5
2	2	1.5
3	3	4
4	4	4
5	5	5.5
6	6	7
7	7	7
8	8	8.5

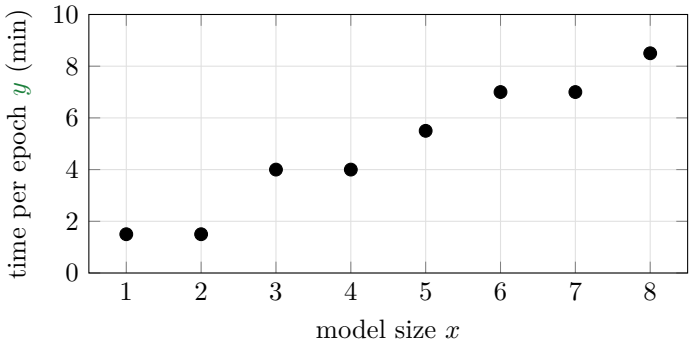


Figure 1. The eight points.

The weak learner. A **tree with two levels of questions**: the first question splits the points in two groups; each group is split once more. It has up to four **leaves**; each leaf predicts the *average of* y of its points (where heavier points count more). We choose each question by trying every threshold $(1.5, 2.5, \ldots, 7.5)$ and keeping the one that makes the *weighted squared error* smallest.

2. Round 1, step by step

1 Step 1. Every point starts with the same weight. $w_i = \dfrac{1}{8} = 0.125$. The weights always add up to 1.

2 Step 2. Grow the tree. First question: try all seven thresholds. For each, the two groups predict their averages and we compute the weighted squared error $\sum_i w_i (y_i - h(x_i))^2$:

question	points in each group	predictions	weighted squared error	
$x \le 1.5$	1 / 7	1.50 / 5.36	4.232	
$x \le 2.5$	2 / 6	1.50 / 6.00	2.062	
$x \le 3.5$	3 / 5	2.33 / 6.40	1.983	
$x \le 4.5$	4 / 4	2.75 / 7.00	1.344	**smallest**
$x \le 5.5$	5 / 3	3.30 / 7.50	1.725	
$x \le 6.5$	6 / 2	3.92 / 7.75	3.104	
$x \le 7.5$	7 / 1	4.36 / 8.50	3.982	

The first question is $x \le 4.5$. Then each group is split once more in the same way: the left group (points 1–4) by $x \le 2.5$, the right group (points 5–8) by $x \le 5.5$. The tree and its four leaves:

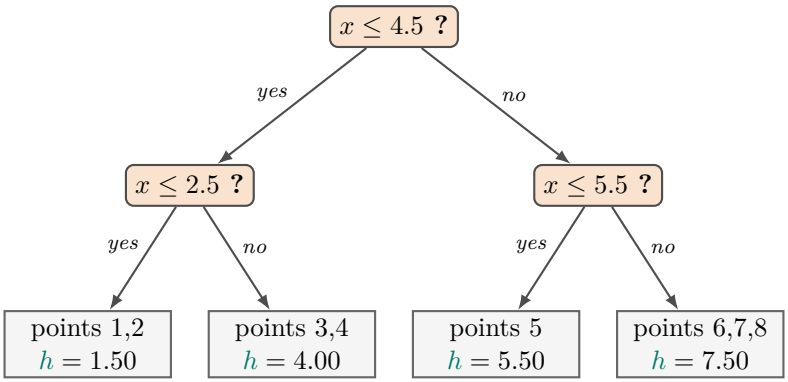


Figure 2. The tree of round 1. Each leaf shows its points and its prediction h, the average of y in the leaf.

Your turn 1: a leaf value

The right-hand leaf of the second level (points 6, 7, 8) predicts the average of their y (all weights are equal). Compute it. Then compute the error $\delta_i = |y_i - h(x_i)|$ of each of the points 5, 6, 7, 8.

Hint: $y_5 = 5.5$ *is alone in its leaf;* $y_6 = 7$, $y_7 = 7$, $y_8 = 8.5$.

3 Step 3. How wrong is the tree on each point? The **error** of a point is $\delta_i = |y_i - h(x_i)|$. Let Δ be the *largest* error in this round. The **relative loss** of a point is

$$\boxed{\ell_i = \frac{\delta_i}{\Delta}} \qquad 0 \text{ (perfect)} \le \ell_i \le 1 \text{ (the worst mistake of the round).}$$

(Other choices are possible: squaring ℓ_i or using $1 - e^{-\ell_i}$. We use the simplest.)

i	x	y	prediction h	error δ	$\ell = \delta/\Delta$
1	1	1.5	1.50	0.00	0.000
2	2	1.5	1.50	0.00	0.000
3	3	4.0	4.00	0.00	0.000
4	4	4.0	4.00	0.00	0.000
5	5	5.5	5.50	0.00	0.000
6	6	7.0	7.50	0.50	0.500
7	7	7.0	7.50	0.50	0.500
8	8	8.5	7.50	1.00	1.000

Here $\Delta = 1.000$ (points 6, 7 and 8 are off by up to 1).

4 Step 4. The average loss, the shrink factor and the confidence.

$$\bar{\ell} = \sum_i w_i \ell_i = 0.2500, \qquad \boxed{\beta = \frac{\bar{\ell}}{1 - \bar{\ell}}} = \frac{0.25}{0.75} = 0.3333, \qquad \boxed{\alpha = \ln \frac{1}{\beta}} = \ln 3.00 = 1.0986.$$

Reading these. $\bar{\ell}$ is the tree's weighted average relative loss: 0 would be a perfect tree, $\frac{1}{2}$ is the borderline. β is a number below 1 whenever $\bar{\ell} < \frac{1}{2}$; the *smaller* the loss, the smaller β. The confidence α is the opposite: a small loss gives a large α, a louder voice in the final prediction. (If $\bar{\ell} \ge \frac{1}{2}$ the tree is no better than a coin toss on average, so $\beta \ge 1$ and the algorithm stops.)

Your turn 2: the average loss and the voice

Using the table, with all weights equal to $\frac{1}{8}$, compute $\bar{\ell}$. Then β and α. Use $\ln 3 = 1.0986$.

Hint: only points 6, 7, 8 have a non-zero ℓ: add them, then divide by 8.

5 Step 5. Change the weights. Every weight is multiplied by $\beta^{1-\ell_i}$, and then everything is divided by the total Z so that the weights add up to 1 again:

$$w_i \leftarrow \frac{w_i\,\beta^{1-\ell_i}}{Z}, \qquad Z = \sum_i w_i \beta^{1-\ell_i}.$$

Read the exponent. A point predicted **perfectly** ($\ell = 0$) is multiplied by $\beta^1 = \beta < 1$: it shrinks. A point with the **worst** error ($\ell = 1$) is multiplied by $\beta^0 = 1$: it stays. Points in between shrink a little. After dividing by Z, the badly predicted points carry *relatively more* weight than before.

i	w before	ℓ	multiplier $\beta^{1-\ell}$	w·multiplier	w after ($\div Z$)
1	0.1250	0.000	0.3333	0.0417	0.0872
2	0.1250	0.000	0.3333	0.0417	0.0872
3	0.1250	0.000	0.3333	0.0417	0.0872
4	0.1250	0.000	0.3333	0.0417	0.0872
5	0.1250	0.000	0.3333	0.0417	0.0872
6	0.1250	0.500	0.5774	0.0722	0.1511
7	0.1250	0.500	0.5774	0.0722	0.1511
8	0.1250	1.000	1.0000	0.1250	0.2617
			totals: Z_1 and 1	0.4777	1.0000

Your turn 3: re-weight two points

With $\beta = \frac{1}{3}$ and $Z_1 = 0.4777$, compute the new weight of point 1 (its $\ell = 0$) and of point 8 ($\ell = 1$). Each starts at 0.125.

Hint: multiplier $= \beta^{1-\ell}$: for $\ell = 0$ it is β, for $\ell = 1$ it is 1. Multiply, then divide by Z_1.

6 Step 6. What just happened? Point 8 (the worst error) went from 0.125 to 0.262: more than double. Points 1–5, which the tree predicted exactly, fell to 0.087 or 0.087. The next tree is asked to minimise the *weighted* squared error, so it will try harder on the heavy points 6, 7, 8.

3. Rounds 2 to 6: all the trees

The recipe repeats: grow the tree on the current weights, compute ℓ_i, $\bar{\ell}$, β, α, then re-weight. Figure 3 shows **every tree**. Notice they differ: the weights change which question is best and what each leaf's weighted average is.

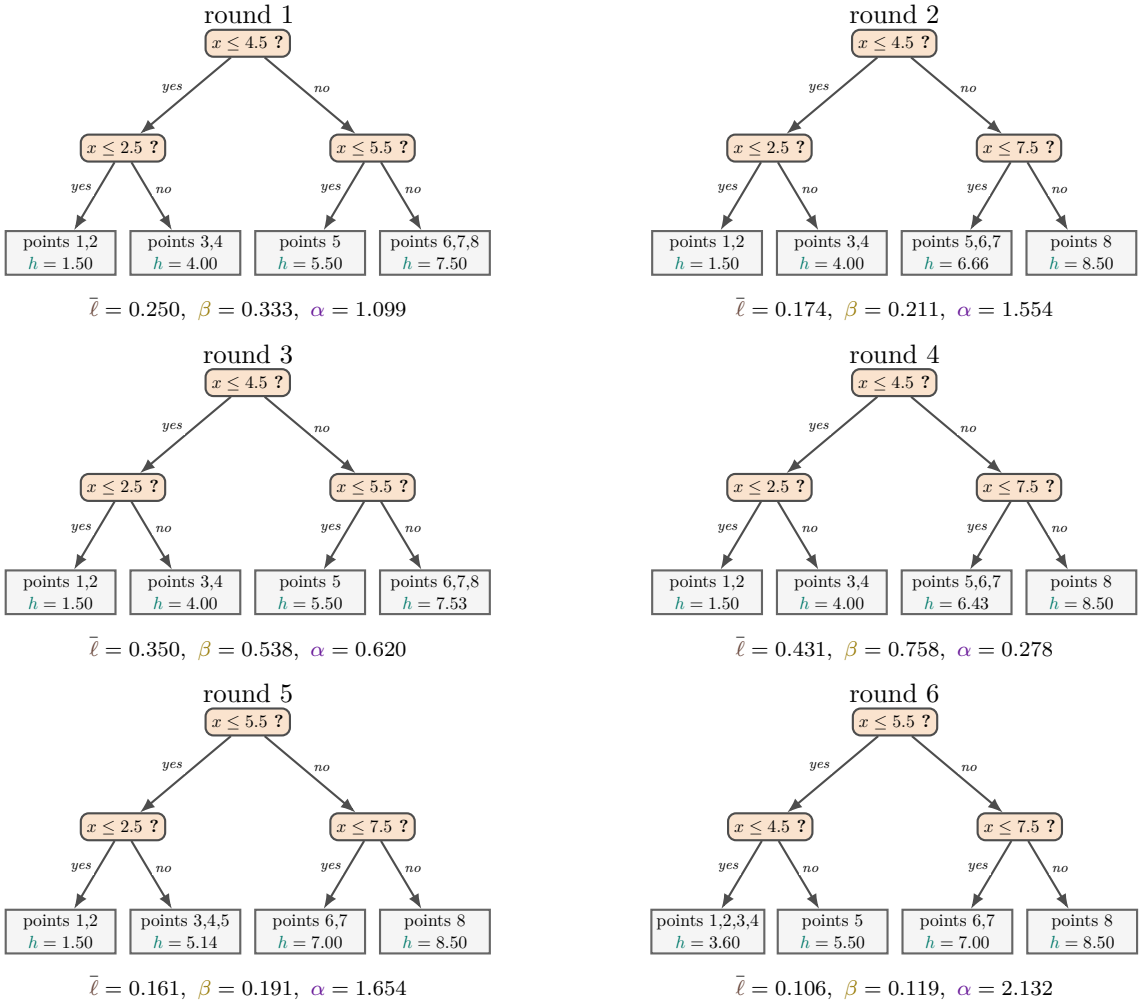


Figure 3. The six trees. Each leaf shows its points and its prediction (weighted average of y).

round	Δ	$\bar{\ell}$	β	$\alpha = \ln(1/\beta)$	Z
1	1.000	0.2500	0.3333	1.0986	0.4777
2	1.164	0.1745	0.2113	1.5544	0.3163
3	0.966	0.3497	0.5377	0.6204	0.6881
4	0.927	0.4310	0.7576	0.2776	0.8593
5	1.136	0.1606	0.1913	1.6537	0.2898
6	2.098	0.1061	0.1186	2.1317	0.1819

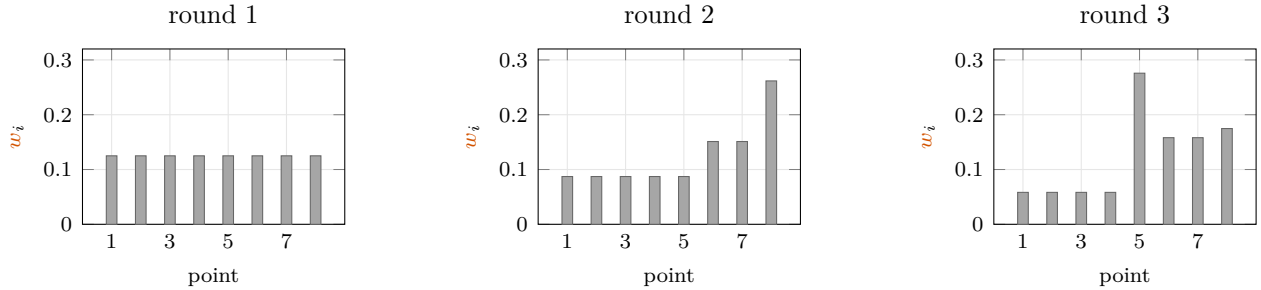


Figure 4. The weights each round starts with. Round 1 is flat. After round 1 point 8 is the tallest; after round 2 the weight moves to point 5.

4. The final prediction: all the trees together

We do not pick one tree, and we do not simply add or average them. To predict at a point x:
1. **Every one** of the T trees gives its own number $h_t(x)$.
2. Each tree carries a number of **votes** equal to its confidence α_t (a tree with a small loss has many votes).
3. The final prediction is the value that has half of all the votes at or below it: the **weighted median**.

First, the ordinary median. If three trees say 6.4, 7.0 and 7.5, then in sorted order the middle one is 7.0. A weighted median is the same idea when some trees count for more than others: instead of finding the middle *tree*, we find the middle *vote*.

The simplest case: two trees, and what α does. At $x = 6$, after round 2 there are only two trees. Tree 1 says 7.50 and has $\alpha_1 = 1.099$ votes. Tree 2 says 6.66 and has $\alpha_2 = 1.554$ votes. The total is 2.653 votes, so half is 1.327. In sorted order tree 2 (6.66) comes first and already has $1.554 \geq 1.327$, so the final prediction is **6.66**. If tree 1 had been the more confident of the two (say $\alpha_1 = 2$ against $\alpha_2 = 1$), the same rule would have given 7.50 instead. So the answer is *one tree's number*, as every median is one of the numbers, but **which tree's number it is is decided by the confidences**: the more confident tree wins. With more trees the same thing happens, only the votes of all trees are counted to decide whose number lands on the halfway line.

How to find it: the running total. Sort the trees by what they predict, smallest first. Go down the list and keep a running total of the votes counted so far (add each tree's α_t to the total of the rows above it). The answer is the prediction in the **first row where the running total reaches half of all the votes**.

Worked example at $x = 6$ (six trees). Total votes = 7.336, half = 3.668.

tree (round)	prediction $h_t(6)$	confidence α_t	running total	
4	6.43	0.278	0.278	
2	6.66	1.554	1.832	
5	7.00	1.654	3.486	
6	7.00	2.132	5.617	← **median**
1	7.50	1.099	6.716	
3	7.53	0.620	7.336	

The running total first reaches 3.668 at the prediction 7.00, so the weighted median at $x = 6$ is **7.00** minutes (the truth is 7).

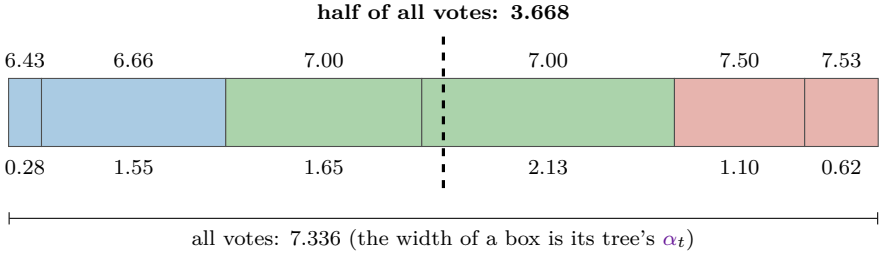


Figure 5. The six trees sorted by prediction (the number in each box); the small number under a box is its votes α_t. The dashed line marks half of all votes; it falls inside the green box, so its prediction is the weighted median.

Why a median and not an average? A single bad tree, even a confident one, can give a wild value; a median is not dragged far by it. Two trees say 7.5 (rounds 1 and 3), two say 6.66 (rounds 2 and 4), two say 7.0: the median settles at 7.0, in the middle of them.

The three combination rules side by side.

	Every tree gives	**The final answer is**
Gradient boosting (Parts 1, 2)	a correction $\nu\gamma$	the **sum** of all the corrections, added to the start
AdaBoost, classification (Part 4)	a vote $+1$ or -1	the **sign of the weighted sum** $\sum \alpha_k h_k$
AdaBoost, regression (this part)	a number $h_t(x)$	the **weighted median** of the numbers, with weights α_t

Your turn 4: a weighted median

At $x = 5$ the six trees give: round 1: 5.50, round 2: 6.66, round 3: 5.50, round 4: 6.43, round 5: 5.50, round 6: 5.50, with the confidences α of the table above (1.099, 1.554, 0.620, 0.278, 1.654, 2.132). Sort, add the confidences, find the median.

Hint: total $\alpha = 7.336$, half $= 3.668$. Which prediction occurs with the largest total confidence?



Figure 6. The ensemble's prediction (red) against the data. One tree is already close; tree 2 makes points 5–7 worse (all three predicted 6.66); later trees, with more confidence on the points that were wrong, pull the median back until all eight points are matched.

trees	$x=1$	$x=2$	$x=3$	$x=4$	$x=5$	$x=6$	$x=7$	$x=8$
1	1.50	1.50	4.00	4.00	5.50	7.50	7.50	7.50
2	1.50	1.50	4.00	4.00	6.66	6.66	6.66	8.50
3	1.50	1.50	4.00	4.00	5.50	7.50	7.50	7.53
4	1.50	1.50	4.00	4.00	6.43	6.66	6.66	8.50
5	1.50	1.50	4.00	4.00	5.50	7.00	7.00	8.50
6	1.50	1.50	4.00	4.00	5.50	7.00	7.00	8.50
truth y	1.50	1.50	4.00	4.00	5.50	7.00	7.00	8.50

Note. On these eight points the ensemble ends up matching the data exactly. Real data are noisy; there the ensemble would *not* match every point, and it should not. The number of rounds is chosen on separate validation data, not by watching the training error go to zero.

5. Derive it yourself

AdaBoost.R2 is a practical extension of the classification AdaBoost; it is not derived from one loss in a few lines. The boxes below show what *can* be derived, and in particular that it contains the classification AdaBoost as a special case. Work through them in order; the full working is in Section 8.

Box D1: the classification AdaBoost hides inside

Suppose every point is either predicted perfectly ($\ell_i = 0$) or is a worst mistake ($\ell_i = 1$), like right/wrong in classification, and let ε be the total weight of the wrong points. (a) What are $\bar{\ell}$ and β? (b) By what factor is a right point multiplied, by what a wrong point? (c) Show that the ratio (wrong weight)/(right weight) after the update equals $\dfrac{1-\varepsilon}{\varepsilon}$, exactly the ratio of classification AdaBoost's factors e^{α} and $e^{-\alpha}$ with $\alpha = \frac{1}{2}\ln\frac{1-\varepsilon}{\varepsilon}$.

Hint: with only $\ell \in \{0,1\}$, $\bar{\ell} = \sum w_i \ell_i = \varepsilon$; the multiplier is $\beta^{1-\ell}$.

Box D2: the confidence

Show that $\ln(1/\beta)$ equals $2\alpha_{\text{class}}$, twice the vote weight $\alpha_{\text{class}} = \frac{1}{2}\ln\frac{1-\varepsilon}{\varepsilon}$ of classification AdaBoost. Does doubling every confidence change the weighted median or the weighted vote?

Hint: $\beta = \frac{\varepsilon}{1-\varepsilon}$ in the special case of D1. Doubling every weight keeps "reaches half of the total".

Box D3: why the weighted median

Let the trees predict $v_1 \le v_2 \le \cdots \le v_T$ with confidences α_t. Consider $S(m) = \sum_t \alpha_t |m - v_t|$ (the total weighted distance from a candidate prediction m). Show that S is smallest at the weighted median.

Hint: if m moves right by a tiny d, each v_t to the left becomes farther by d and each v_t to the right becomes closer by d. When does moving right stop helping?

Box D4: when to stop

Show that $\beta < 1$ exactly when $\bar{\ell} < \frac{1}{2}$, and that then $\alpha = \ln(1/\beta) > 0$. What does $\bar{\ell} \ge \frac{1}{2}$ mean for the tree, and why is it sensible to stop?

Hint: $\beta = \frac{\bar{\ell}}{1-\bar{\ell}} < 1 \iff \bar{\ell} < 1 - \bar{\ell}$.

Box D5: what the multiplier does

For $\beta = \frac{1}{3}$ compute the multiplier $\beta^{1-\ell}$ for $\ell = 0$, $\ell = 0.5$ and $\ell = 1$. Which points end up, relatively, with more weight after dividing by Z?

Hint: $\beta^{0.5} = \sqrt{\beta}$.

6. The recipe and common confusions

AdaBoost.R2 in six lines.

1. Give every point the weight $1/N$.
2. Grow a small tree on the weights (leaves = weighted averages of y).
3. Errors $\delta_i = |y_i - h(x_i)|$, relative losses $\ell_i = \delta_i/\Delta$, average loss $\bar{\ell} = \sum w_i \ell_i$ (stop if $\bar{\ell} \ge \frac{1}{2}$).
4. $\beta = \bar{\ell}/(1 - \bar{\ell})$ and confidence $\alpha = \ln(1/\beta)$.
5. $w_i \leftarrow w_i \beta^{1-\ell_i}$, then divide by the total.
6. Repeat; predict the weighted median of the trees' predictions, with weights α_t.

"Why measure the error relative to the worst mistake?" It puts every round on the same scale 0 to 1, so the update works whatever the units of y are. The price: Δ is set by a single point, so one outlier can dominate the scale.

"Why do well-predicted points shrink instead of badly predicted points growing?" Same effect, written the other way round: only the *ratios* of the weights matter after dividing by Z (Box D1).

"Why a median?" It is robust to a tree that is far off, and it is what the classification vote turns into when predictions are numbers (Box D3).

"How does it differ from gradient boosting for regression?" Gradient boosting fits each new tree to the *residuals* and *adds* it (a sum, small steps). AdaBoost.R2 fits each tree to the *original* y with changed weights and *takes a weighted median*.

"Linear, square or exponential loss?" They are three ways to turn the relative error δ/Δ into a loss: $\ell = \delta/\Delta$, $(\delta/\Delta)^2$, $1 - e^{-\delta/\Delta}$. Square punishes large errors more; the choice changes the numbers, not the method.

7. Check yourself

Fill in the blanks without looking, then check against the answers below.

1. Each point starts with weight _____, and the weights always add up to ____.

2. A leaf predicts the _____ of y of its points, where heavier points count _____.

3. The error of a point is $\delta_i =$ _____ and its relative loss is $\ell_i = \delta_i /$ ____.

4. Round 1 has $\Delta =$ _____ and average loss $\bar{\ell} =$ _____.

5. $\beta = \dfrac{\bar{\ell}}{____} =$ _____ and $\alpha = \ln \dfrac{1}{\beta} =$ _____.

6. A point with $\ell = 0$ is multiplied by _____; a point with $\ell = 1$ is multiplied by ____. After dividing by Z the badly predicted points carry relatively _____ weight.

7. After round 1 point 8 has weight _____, and point 1 has weight _____.

8. The final prediction is the _____ of the trees' predictions, with weights _____. At $x = 6$ it is _____.

9. If every point is either perfect ($\ell = 0$) or worst ($\ell = 1$), AdaBoost.R2 gives the same weights as _____ AdaBoost, and $\ln(1/\beta)$ is _____ its vote weight.

10. The algorithm stops when the average loss is at least ____, because then $\beta \geq$ ____.

Answers

1. $1/8 = 0.125$; 1

2. average (weighted mean); more

3. $|y_i - h(x_i)|$; Δ (the largest error in the round)

4. 1; 0.25

5. $1 - \bar{\ell}$; 0.333; 1.099 ($= \ln 3$)

6. β; 1; more

7. 0.262; 0.087

8. weighted median; α_t; 7.00

9. classification (the original, discrete); twice

10. $\frac{1}{2}$; 1

8. Solutions to the boxes

Your turn 1

Leaf of points 6, 7, 8: $\dfrac{7+7+8.5}{3} = \dfrac{22.5}{3} = 7.50$. Errors: point 5: $|5.5-5.5| = 0$; point 6: $|7-7.5| = 0.5$; point 7: $|7-7.5| = 0.5$; point 8: $|8.5-7.5| = 1.0$. The largest error of the round is $\Delta = 1$ (point 8).

Your turn 2

$\ell_6 = \ell_7 = 0.5$, $\ell_8 = 1$, the others 0. $\bar{\ell} = \dfrac{1}{8}(0.5+0.5+1) = \dfrac{2}{8} = 0.25$. $\beta = \dfrac{0.25}{0.75} = \dfrac{1}{3}$. $\alpha = \ln 3 = 1.0986$.

Your turn 3

Point 1 ($\ell = 0$): multiplier $\beta^1 = \frac{1}{3}$; raw $0.125 \times \frac{1}{3} = 0.0417$; new $0.0417/0.4777 = 0.087$. Point 8 ($\ell = 1$): multiplier $\beta^0 = 1$; raw 0.125; new $0.125/0.4777 = 0.262$. (Check: $Z = 5(0.0417) + 2(0.125 \times 0.5774) + 0.125 = 0.2083 + 0.1443 + 0.125 = 0.4777$.)

Your turn 4

Sorted predictions at $x = 5$, with running confidence: 5.14 (round 5, $\alpha = 1.654$, running 1.654); 5.50 (round 3, $\alpha = 0.620$, running 2.274); 5.50 (round 1, $\alpha = 1.099$, running 3.373); 5.50 (round 6, $\alpha = 2.132$, running 5.504); 6.43 (round 4, $\alpha = 0.278$, running 5.782); 6.66 (round 2, $\alpha = 1.554$, running 7.336). Total 7.336, half 3.668. The running total first reaches half at 5.50: the weighted median is **5.50** (the truth is 5.5).

Box D1

(a) With only $\ell \in \{0, 1\}$: $\bar{\ell} = \sum w_i \ell_i = \sum_{\text{wrong}} w_i = \varepsilon$, so $\beta = \dfrac{\varepsilon}{1-\varepsilon}$. (b) A right point ($\ell = 0$) is multiplied by $\beta^1 = \beta$; a wrong point ($\ell = 1$) by $\beta^0 = 1$. (c) After the update the wrong:right weight ratio is $\dfrac{1 \cdot w}{\beta \cdot w} = \dfrac{1}{\beta} = \dfrac{1-\varepsilon}{\varepsilon}$. Classification AdaBoost multiplies wrong points by e^{α} and right ones by $e^{-\alpha}$: ratio $e^{2\alpha} = \dfrac{1-\varepsilon}{\varepsilon}$. The same ratio, so the same weights after dividing by the total.

Box D2

$\ln \dfrac{1}{\beta} = \ln \dfrac{1-\varepsilon}{\varepsilon} = 2 \cdot \frac{1}{2} \ln \dfrac{1-\varepsilon}{\varepsilon} = 2\alpha_{\text{class}}$. Doubling every confidence leaves "the running total reaches half of the total" unchanged, and the sign of a weighted vote unchanged. So AdaBoost.R2's vote is the same as classification AdaBoost's.

Box D3

Sort the v_t. For a m strictly between two neighbouring values, $\dfrac{dS}{dm} = \sum_{v_t<m} \alpha_t - \sum_{v_t>m} \alpha_t$: the confidence on the left minus the confidence on the right. While the left total is smaller than the right total, $dS/dm < 0$ and moving right lowers S; once the left total exceeds the right total, $dS/dm > 0$.

So S is smallest where the left and right confidences balance: at the value where the running total of confidences first reaches half of the grand total, the weighted median.

Box D4

$\beta = \dfrac{\bar{\ell}}{1-\bar{\ell}} < 1 \iff \bar{\ell} < 1 - \bar{\ell} \iff \bar{\ell} < \frac{1}{2}$. Then $\alpha = \ln(1/\beta) > 0$. If $\bar{\ell} \geq \frac{1}{2}$ then $\beta \geq 1$, $\alpha \leq 0$: on average (weighted) the tree's relative loss is at least halfway to the worst possible, so it is no better than a coin toss and adding it with a positive voice would not help; the algorithm stops.

Box D5

$\ell = 0$: $\beta = \frac{1}{3} = 0.333$. $\ell = 0.5$: $\sqrt{1/3} = 0.577$. $\ell = 1$: 1. After the update, a perfectly predicted point carries only a third of its former weight (before dividing by Z) while a worst-predicted point keeps all of it, so after dividing by Z the badly predicted points have the larger share.